

Beginner's Guide to C++ Audio Programming with the JUCE Framework

C++ Online Conference Workshop by Jan Wilczek

Introduction

- Jan Wilczek [Yan Vil-check]
- Audio programming consultant & coach
- Founder of TheWolfSound.com blog & YouTube channel
- WolfTalk podcast host
- Online course creator
 - DSP Pro on digital audio signal processing
 - Official JUCE C++ framework audio plugin development course



Rules

- Workshop repo: <https://github.com/JanWilczek/cpp-online-26-workshop>
- These slides: *docs/slides/cpponline-2026-workshop-slides.md*
- Task descriptions: *docs/tasks/tasks.md*
- Please, have your camera turned on, if possible
 - If you need a break, or some time in quiet, just turn off your camera; we'll know you're not there
- We won't split into break rooms, as there's only 7 of us
- Always ask, when in doubt: feel free to interrupt me
- It's ok not to understand the entire code
- I am here to help you and depending on the needs, I will want to help everyone
- I may skip some of the tasks if we're short on time

Prerequisite: Task 0

docs/tasks/task0.md

Part 1: Building a music player in C++

Task: Play back a sine

```
git checkout task/generate-sine
```

Focus: *music_player/source/main.cpp*

We'll be using the PortAudio library by Ross Bencina, Phil Burk, et al.

Task: Play back a sine

Initializing PortAudio

```
const auto error = Pa_Initialize();  
//...  
if (error == paNoError) {  
    Pa_Terminate();  
}
```

Task: Play back a sine

Initializing PortAudio

```
class MusicPlayer {  
    //...  
    pa_ex::Initializer initializer_;  
};
```

Task: Play back a sine

Creating a stream

```
PaStream* stream;  
const auto error = Pa_OpenDefaultStream( &stream, /* many arguments */);  
//...  
if (stream != nullptr && error == paNoError) {  
    Pa_CloseStream(stream_);  
}
```

Task: Play back a sine

Creating a stream

```
class MusicPlayer {  
explicit MusicPlayer()  
    : stream_{/* many arguments */} {}  
    //...  
    pa_ex::Stream stream_;  
};
```

Task: Play back a sine

Generating the sine

$$s[n] = A \sin(2\pi f n / f_s),$$

where

- n is the unitless sample index
- $s[n]$ is the discrete output signal
- f is the frequency in Hz
- f_s is the sample rate
- A is the unitless amplitude
- $\pi = 3.14159 \dots$

Task: Play back a sine

Generating the sine

$$s[n] = A \sin(2\pi f n / f_s),$$

```
auto phase_ = 0.f;
//...
constexpr auto amplitude = 0.25f;
const auto outputSample = amplitude * std::sin(phase_);

// output the sample...

constexpr auto frequency = 220.f;
phase_ += 2 * std::numbers::pi_v<float> * frequency / sampleRate_;
```


Task: Play back an audio file

```
git checkout task/file-player
```

Focus:

- *fx/include/fx/fx.h*
- *music_player/source/main.cpp*

We'll be using the AudioFile library by AdamStark.

Task: Flanger audio effect

```
git checkout task/flanger-effect
```

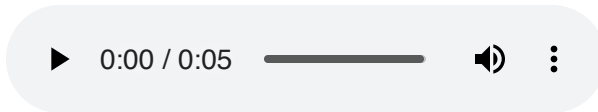
Focus:

- *fx/include/fx/fx.h*
- *music_player/source/main.cpp*

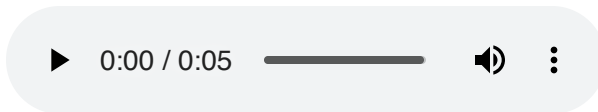
Task: Flanger audio effect

What is Flanger?

Input:



Output:



Task: Flanger audio effect

What is flanger?

A DSP algorithm that can be depicted as:

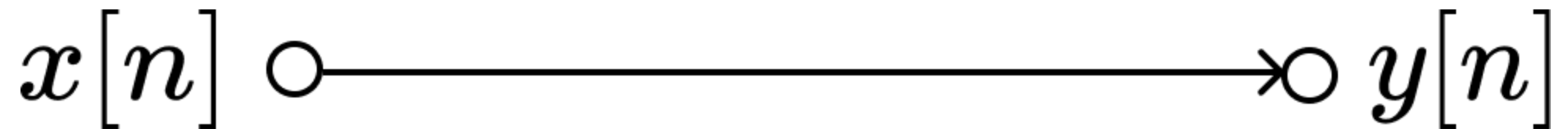
- DSP diagram
- Difference equations
- Textual description
- Literature reference
 - "We will implement equations X-Y from paper Z"

Sources of DSP algorithms

- Books
- Research papers
- Online blogs & videos
- Online forums
- External consultancy
- Self-design (e.g., through experiments)

How to read DSP diagrams?

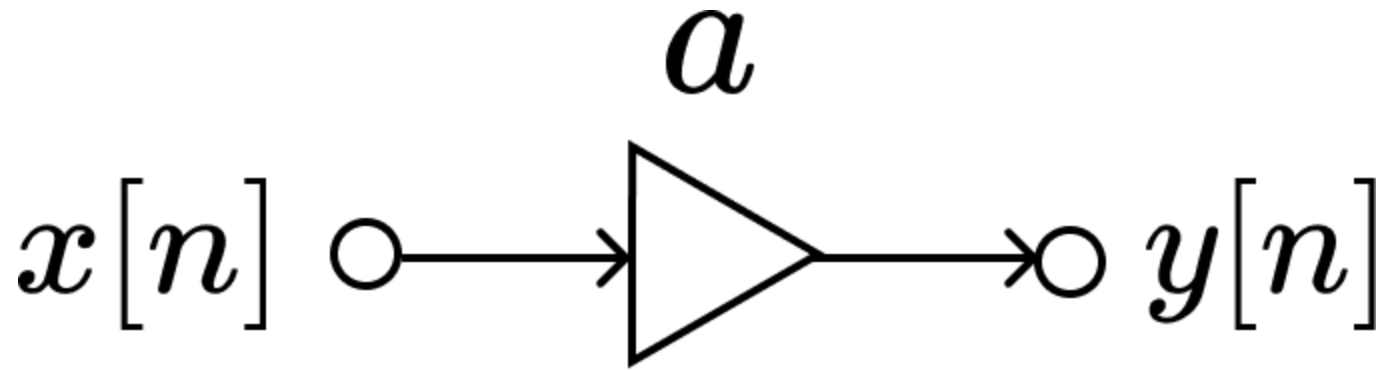
Identity



$$y[n] = x[n]$$

How to read DSP diagrams?

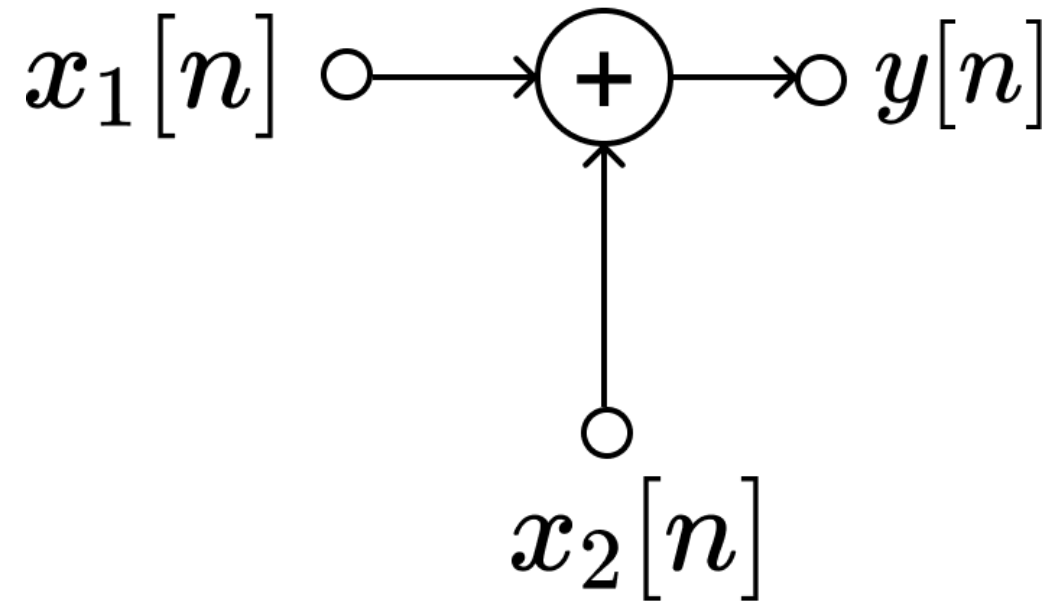
Multiplication



$$y[n] = ax[n]$$

How to read DSP diagrams?

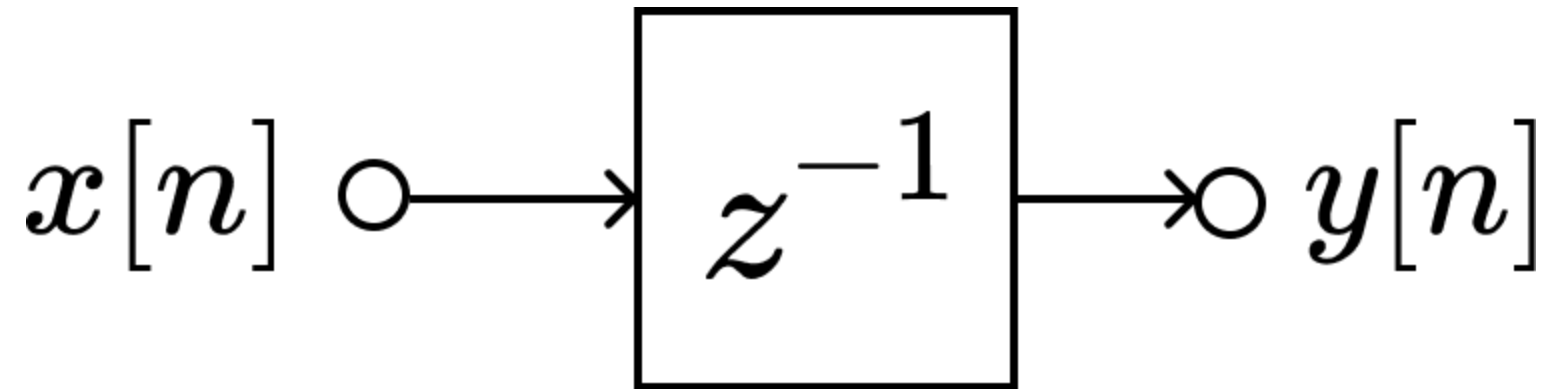
Addition



$$y[n] = x_1[n] + x_2[n]$$

How to read DSP diagrams?

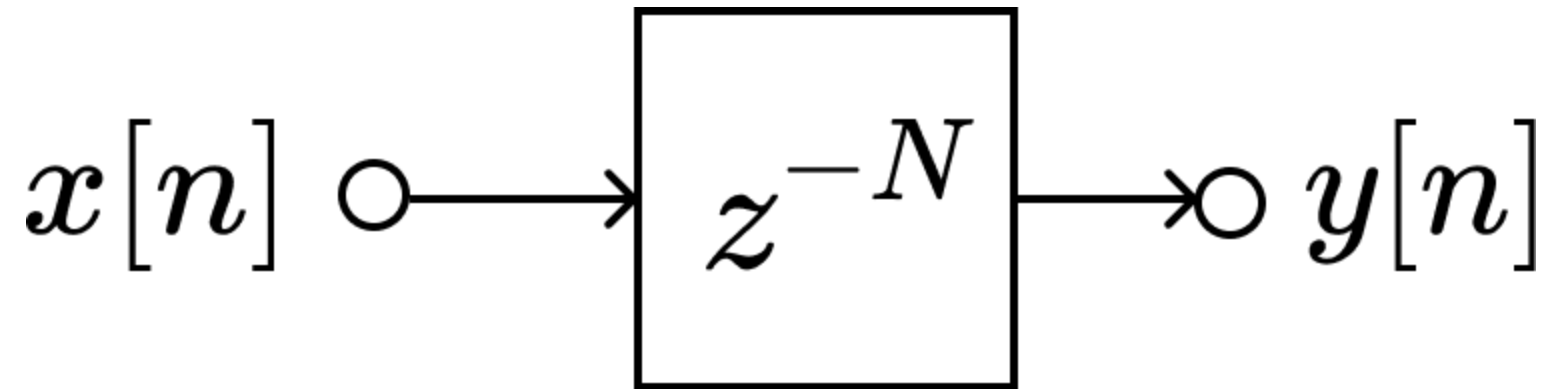
Delay by 1 sample



$$y[n] = x[n - 1]$$

How to read DSP diagrams?

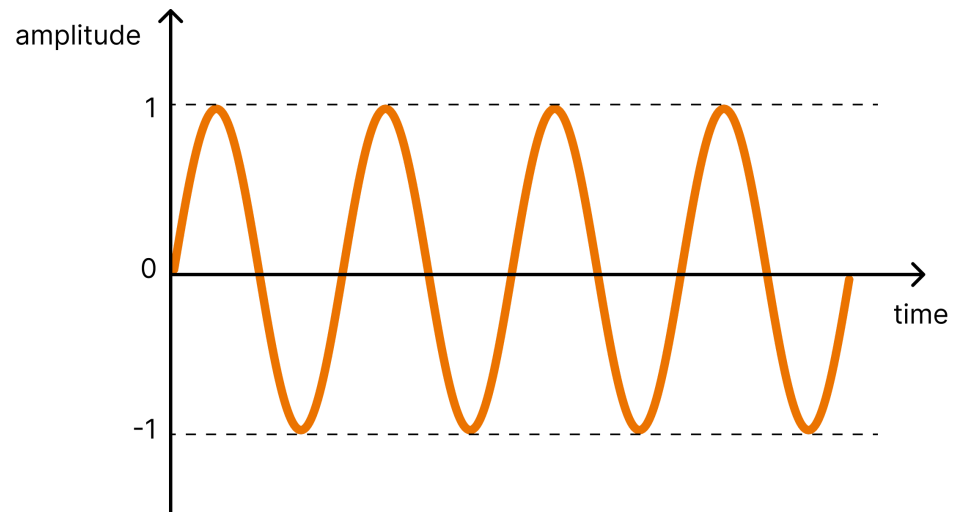
Delay by N samples



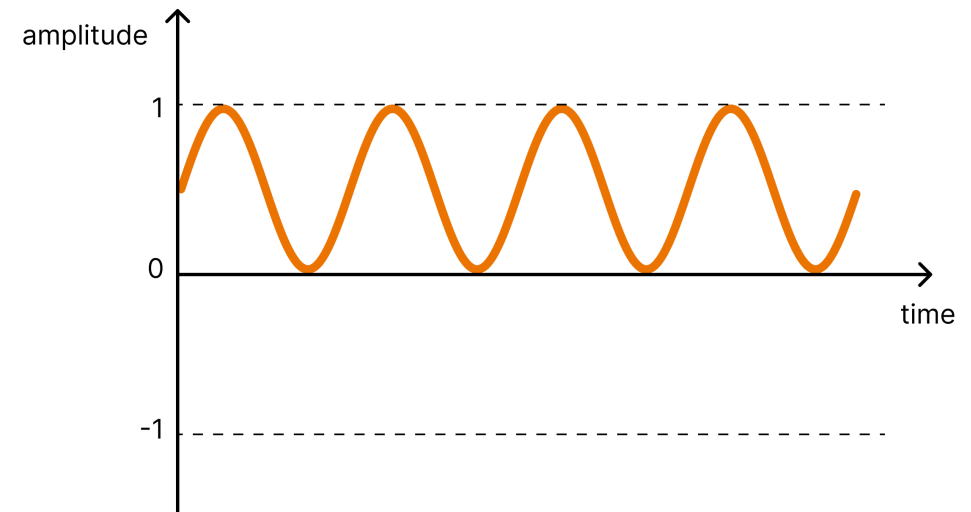
$$y[n] = x[n - N]$$

Low-frequency oscillator (LFO)

Bipolar

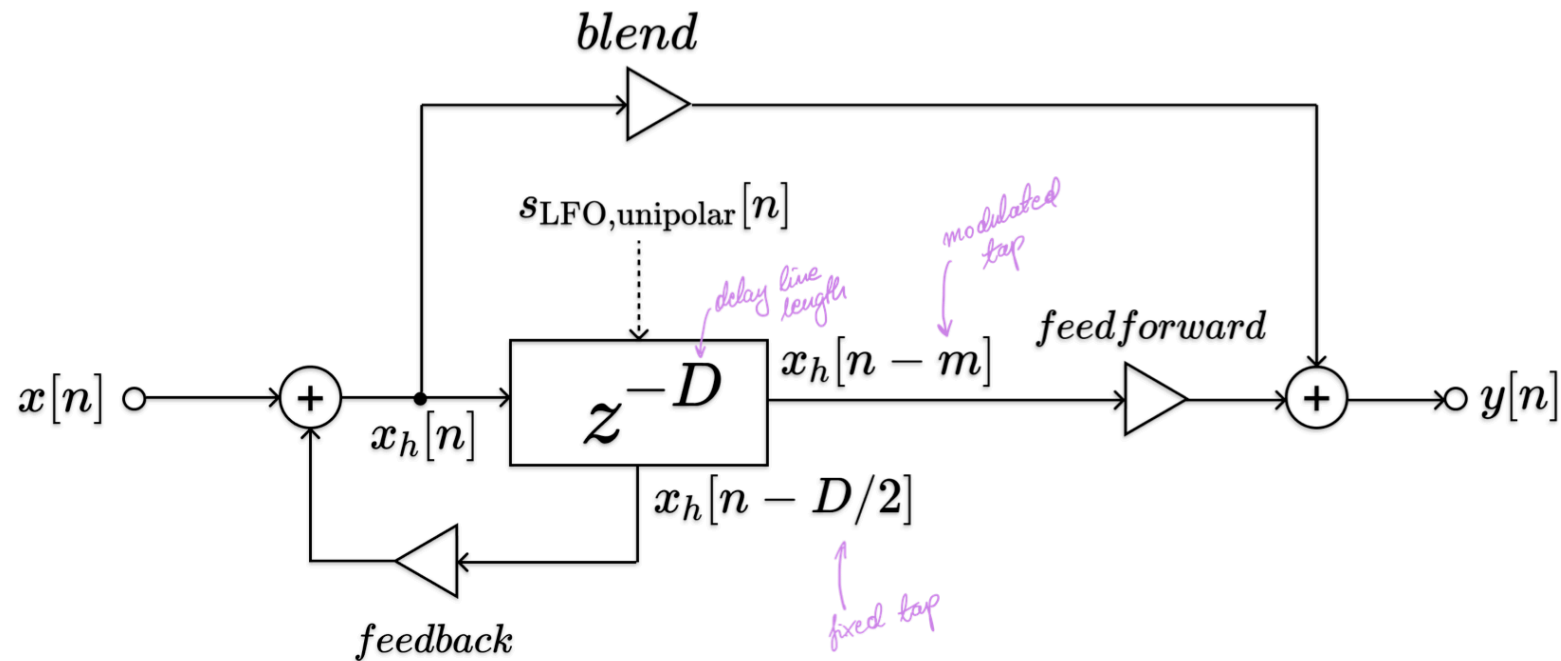


Unipolar



Task: Flanger audio effect

Flanger DSP diagram



Source: J. Dattoro. *Effect design, part 2: Delay-line modulation and chorus*. J. Audio Eng. Soc., 45(10): 764–788, October 1997.

Task: Flanger audio effect

Flanger difference equations

1. Calculate the modulated-delay value

$$m = s_{\text{LFO,unipolar}}[n]D.$$

2. Calculate the helper signal sample

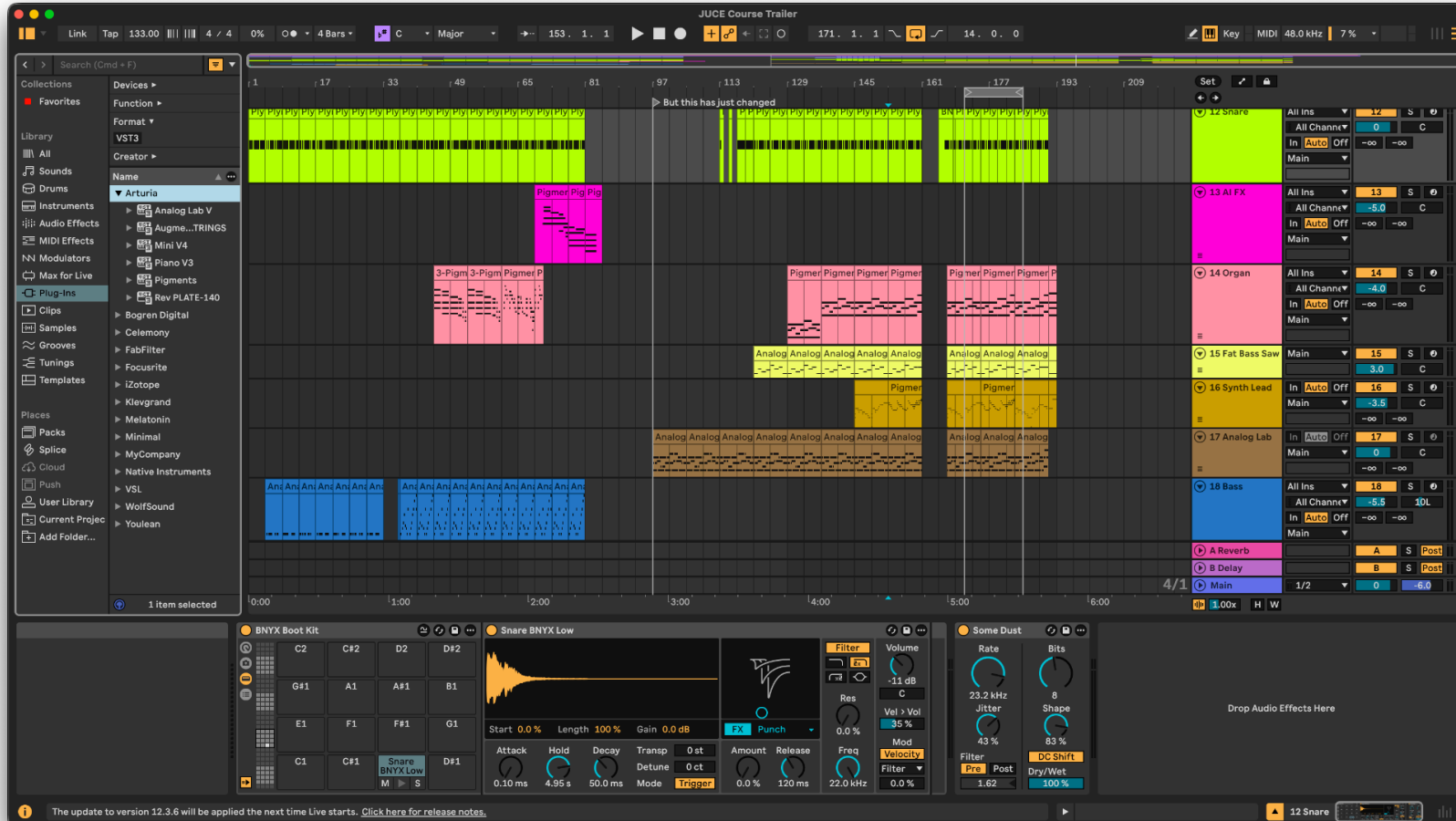
$$x_h[n] = x[n] + \text{feedback } x_h[n - D/2].$$

3. Calculate the output sample

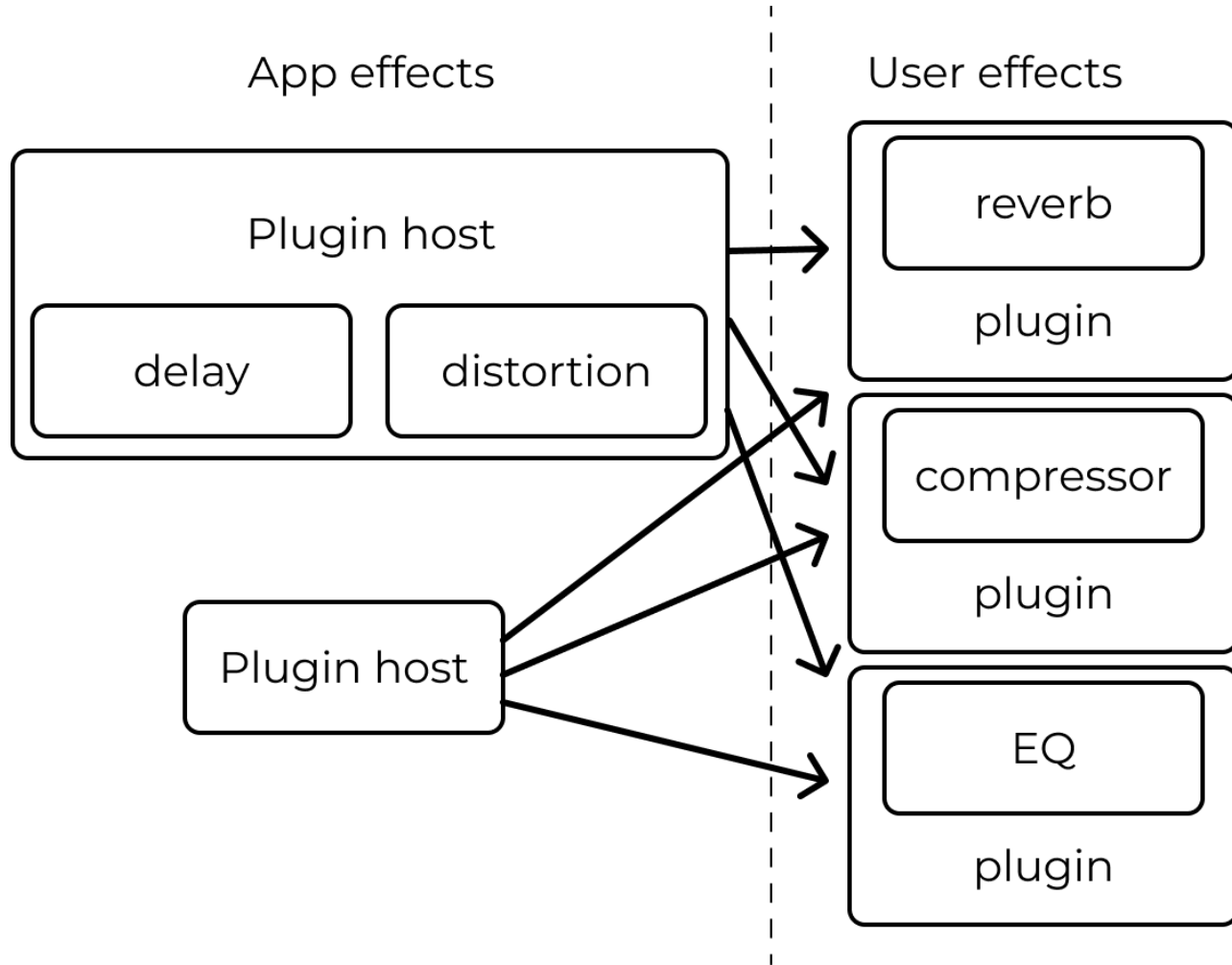
$$y[n] = \text{blend } x_h[n] + \text{feedforward } x_h[n - m].$$

Part 2: Audio plugin in JUCE C++ framework

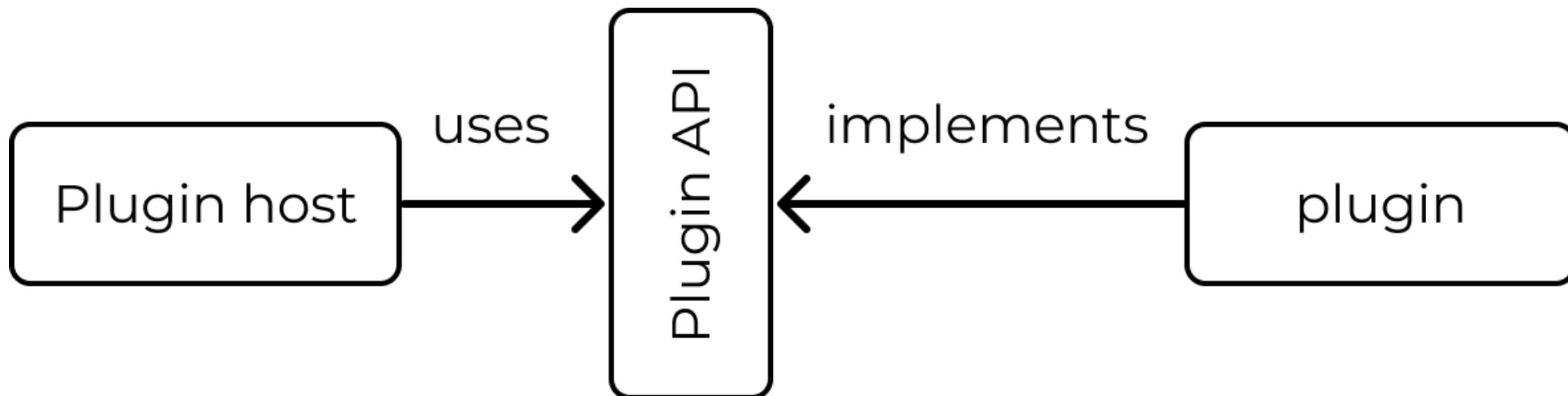
Digital audio workstation (DAW)



DAW plugins



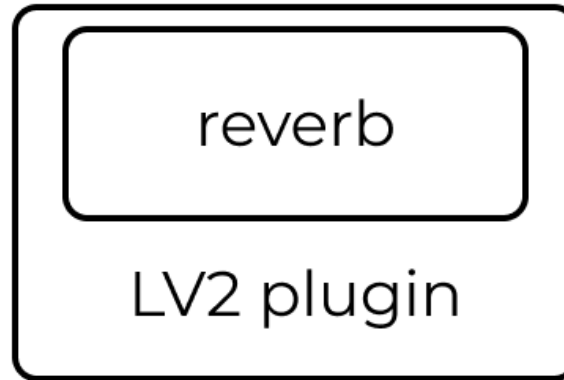
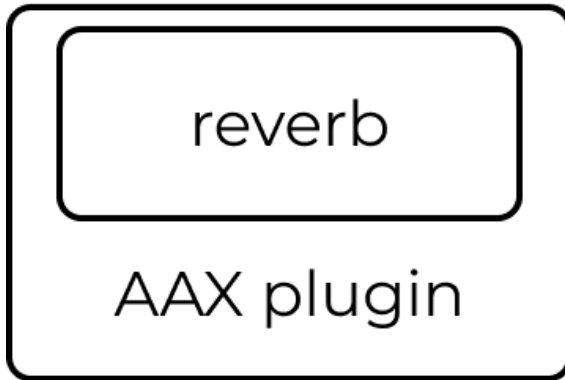
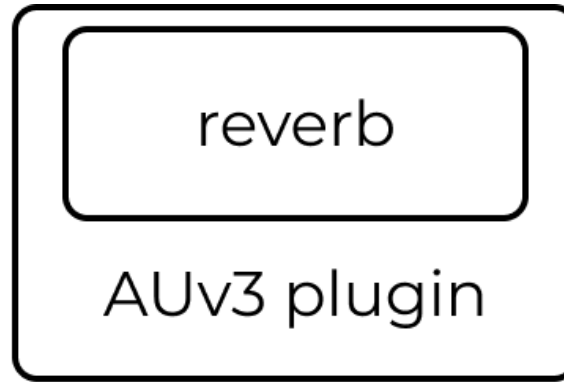
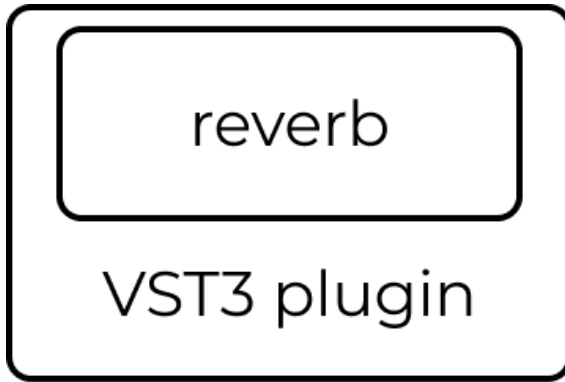
DAW plugins



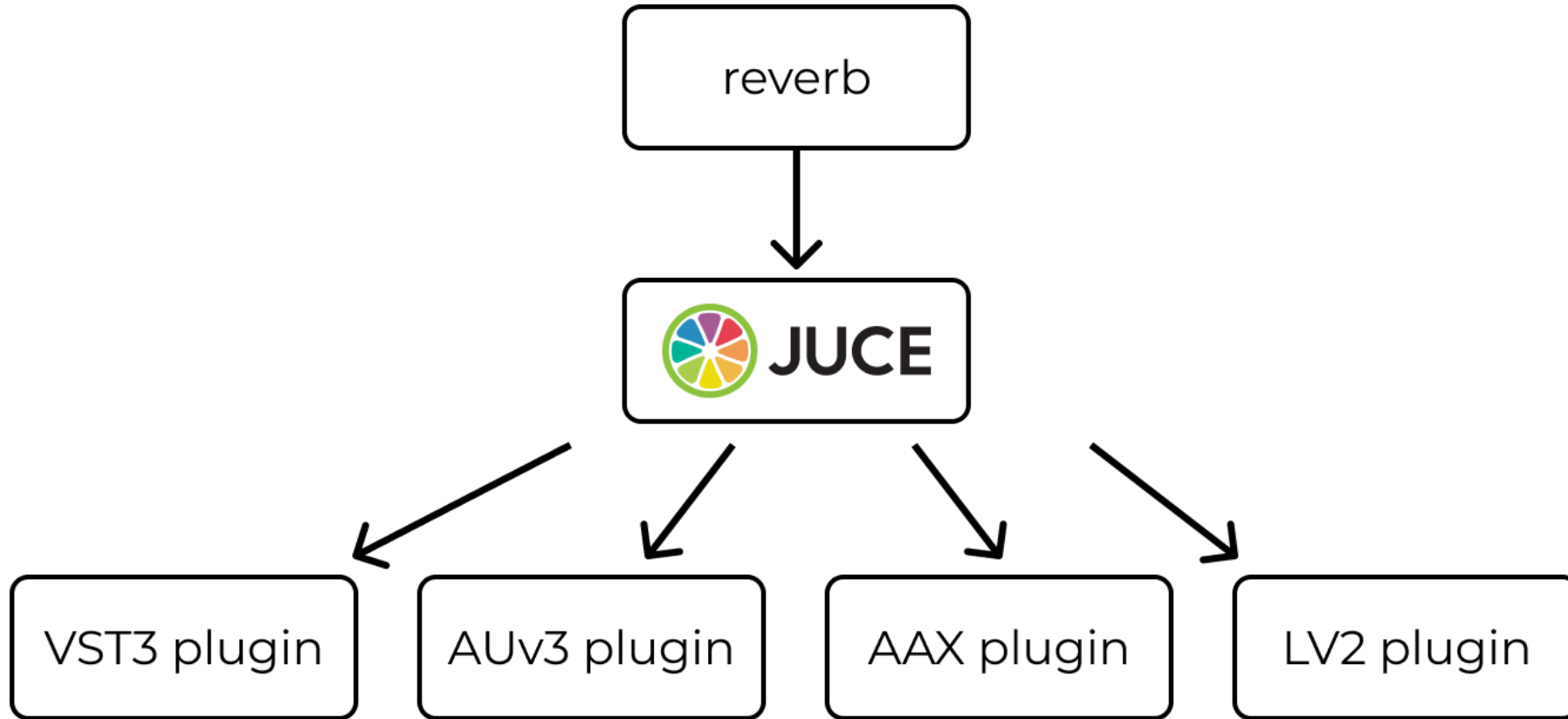
Popular plugin APIs

- Audio Unit v3 (AUv3) by Apple for macOS and iOS
- **Virtual Studio Technology 3 (VST3) by Steinberg**
- Avid Audio eXtensions (AAX) by Avid
- LV2 for Linux
- CLever Audio Plug-in (CLAP)

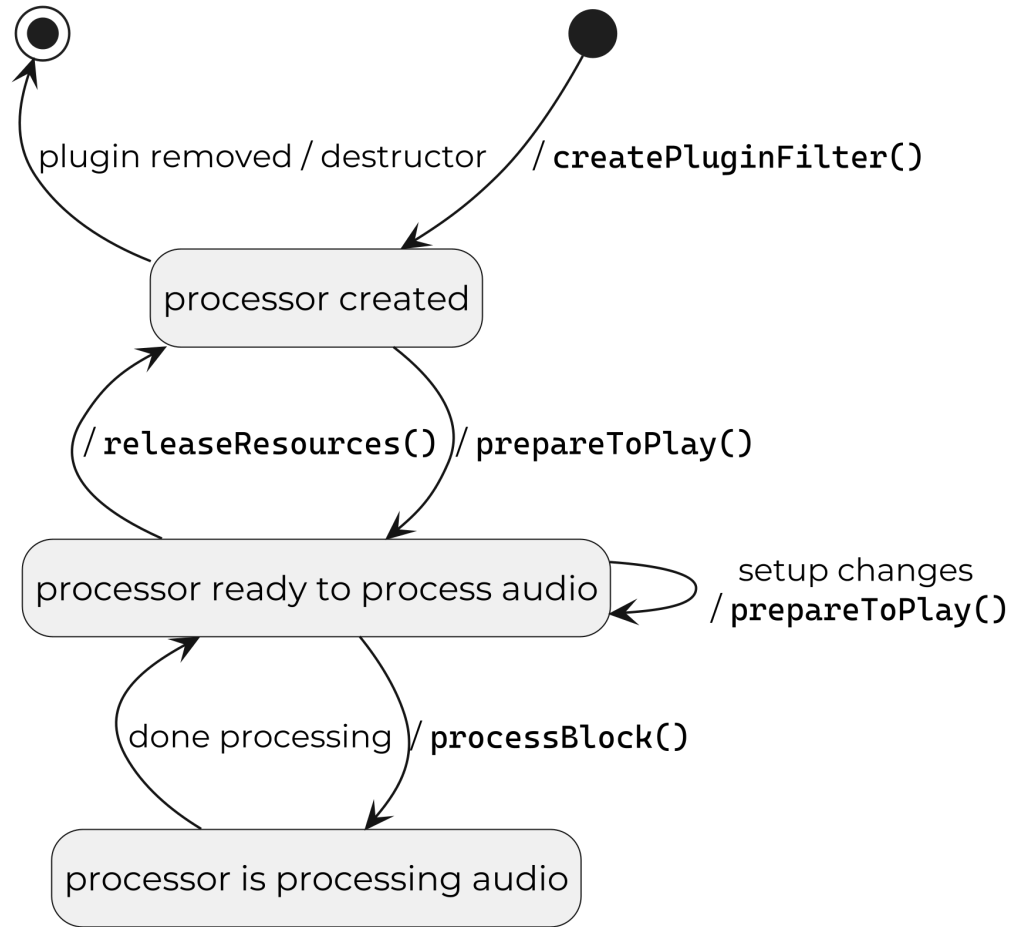
Many plugin formats = development nightmare



Plugin format API abstraction → plugin frameworks



Plugin lifecycle



Task: Flanger plugin

```
git checkout task/flanger-plugin
```

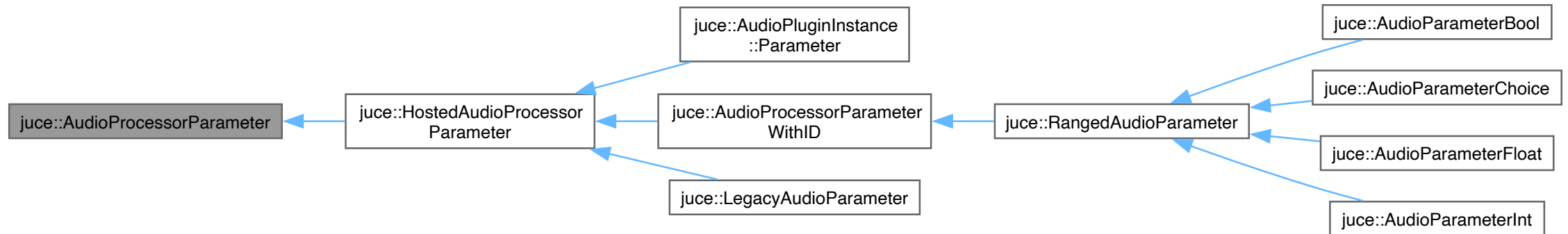
Task: Parameters

```
git checkout task/add-parameter
```

Purpose of plugin parameters

1. DSP control
2. Generic UI
3. Visualization

JUCE parameter classes hierarchy



JUCE parameter classes

- `AudioParameterBool`
 - `true / false`
- `AudioParameterInt`
 - integer in a closed range
 - e.g., "1" from {0, 1, 2}
- `AudioParameterFloat`
 - real value from a closed range
 - e.g., "0.25" from [0, 2]
- `AudioParameterChoice`
 - a value from a fixed set of named options
 - e.g., "lowpass" from {"lowpass", "highpass"}

Purpose of JUCE parameter classes

- Single source of truth
- Plugin version management
- Plugin presets
- GUI thread-audio thread synchronization (explained in more detail later)

JUCE parameter class

Creation (at plugin construction)

- Instantiate dynamically, e.g., using `std::make_unique<>()`
- Retrieve a reference or a pointer
- Call `PluginProcessor::addParameter()`

Usage

- Retrieve the current parameter value in `PluginProcessor::processBlock()`

JUCE parameter class

We'll use my `wolfound::JuceParameterHolder` utility.

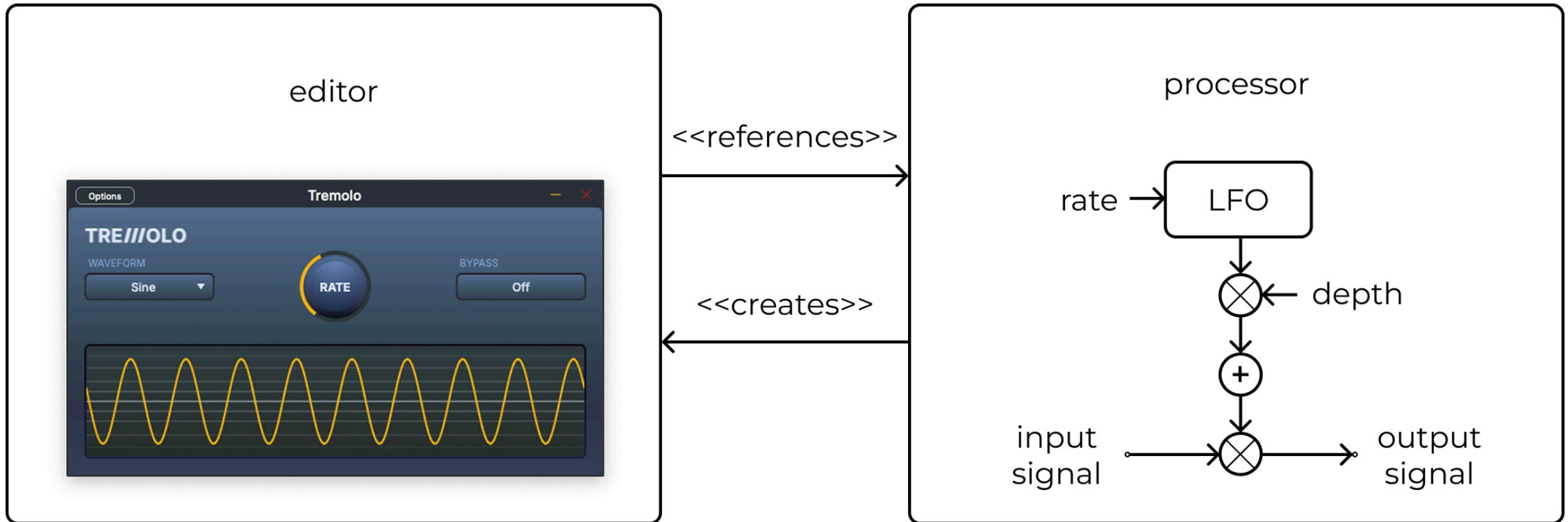
```
struct Parameters {
    juce::AudioParameterFloat& floatParam;
};

class PluginProcessor {
    //...
    Parameters parameters_;
    wolfound::JuceParameterHolder parameterHolder_;
};

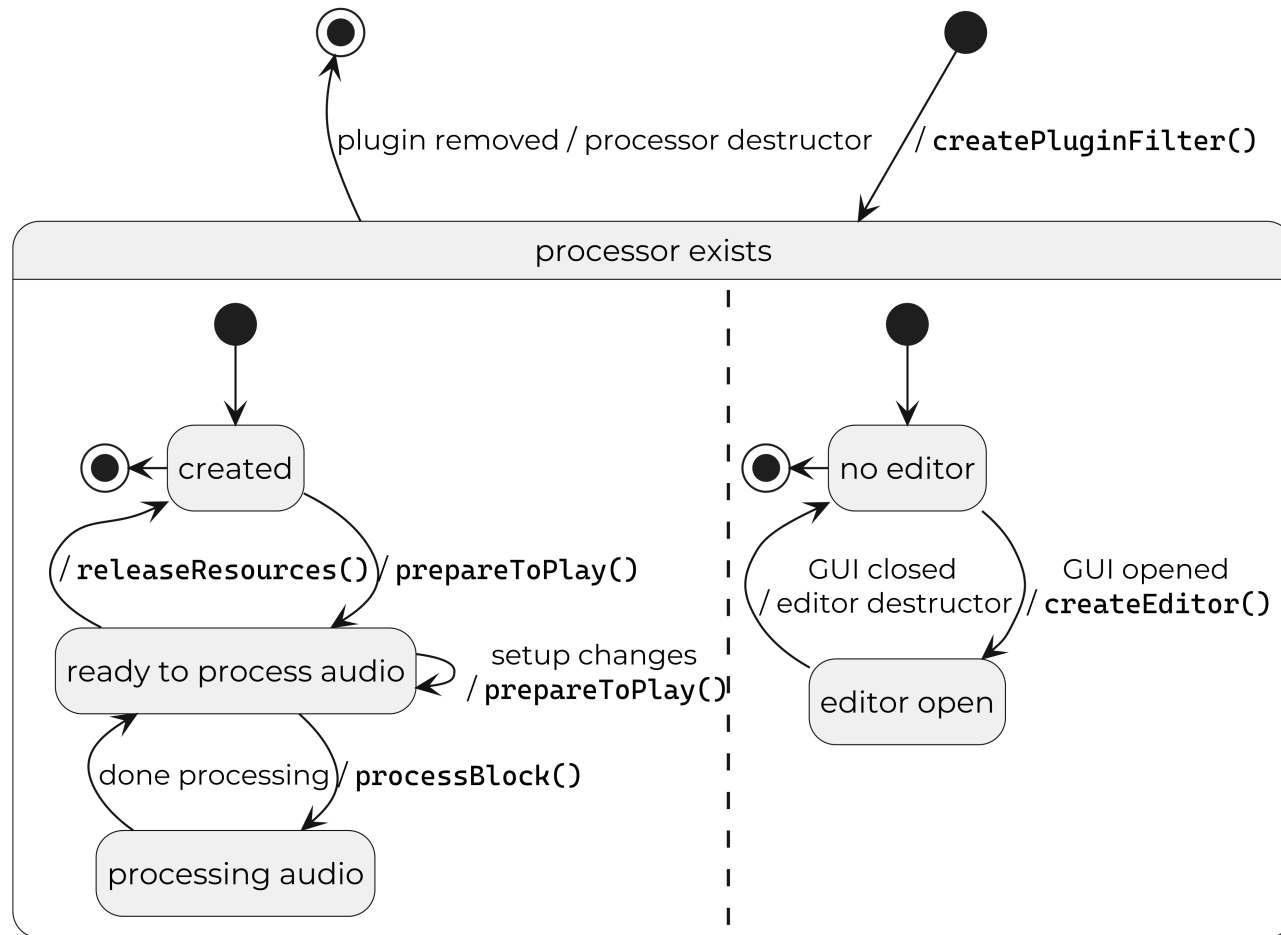
// .cpp file
PluginProcessor::PluginProcessor(
    JuceParameterHolder::Builder builder)
    : parameters_.floatParam{builder.add<juce::AudioParameterFloat>(
        "floatParam",
        "Float Param",
        juce::NormalisableRange{1.f, 10.f},
        5.f)},
        parameterHolder_{std::move(builder).build(*this)} {}
```

Part 3: Plugin GUI in JUCE C++ framework

Processor-editor split



Editor lifecycle



Task: Create a custom editor

```
git checkout task/create-custom-editor
```

Task: Add a slider controlling the modulation rate

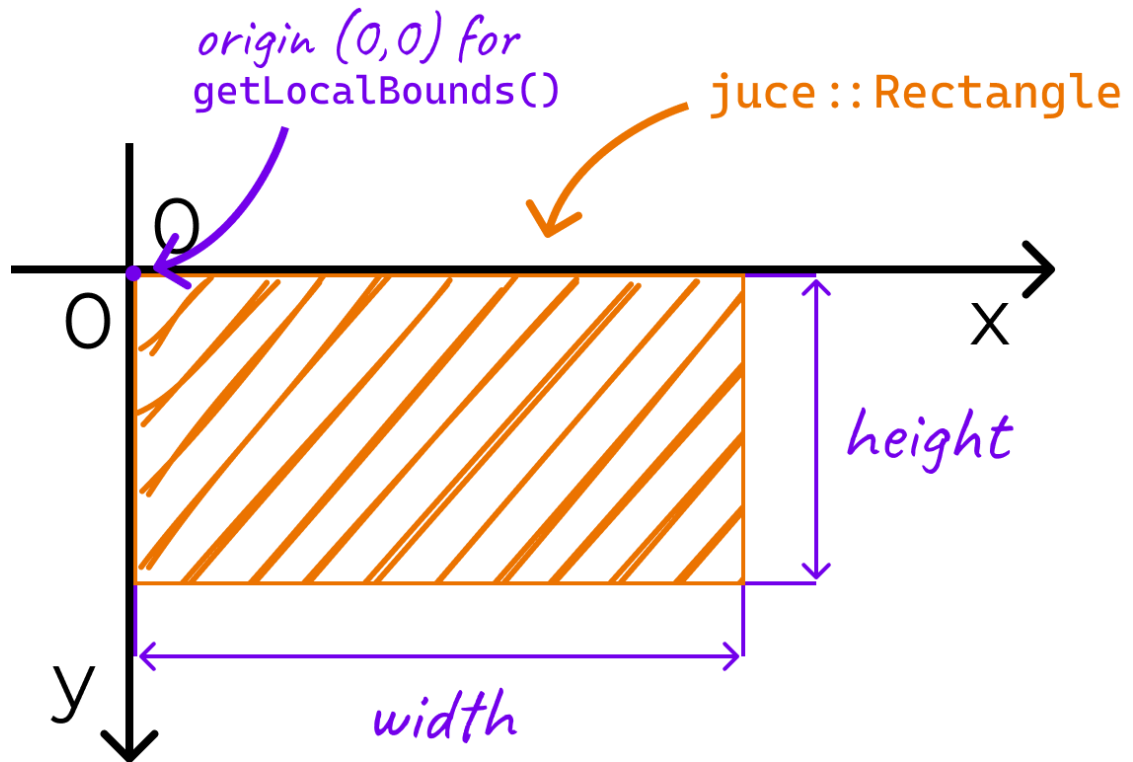
```
git checkout task/add-slider
```



JUCE component system

1. Everything you see on the screen is a `juce::Component` subclass instance
2. Each component has 0, 1, or more child components  component hierarchy

JUCE coordinate system



Adding a new child component

```
class PluginEditor : ... {  
    struct BackgroundComponent : juce::Component {  
  
    };  
    BackgroundComponent background;  
  
public:  
    PluginEditor(...) {  
        addAndMakeVisible(background);  
    }  
  
    void resized() override {  
        background.setBounds(getLocalBounds());  
    }  
};
```

Task: Style components

```
git checkout task/style-components
```

Task: Add labels

```
git checkout task/add-labels
```

Task: Add value label

```
git checkout task/add-value-label
```


Task: Fonts

```
git checkout task/add-custom-fonts
```

JUCE binary data

```
juce_add_binary_data(  
    audio_plugin_assets  
    NAMESPACE  
    audio_plugin::assets  
    SOURCES  
    audio_plugin/assets/MyFont.ttf  
    audio_plugin/assets/SomeImage.png  
)
```

JUCE binary data

```
namespace audio_plugin::assets {  
    extern const char*    MyFont_ttf;  
    const int             MyFont_ttfSize = 47676;  
  
    extern const char*    SomeImage_png;  
    const int             SomeImage_ttfSize = 300764;  
  
    //... some utilities  
}
```

JUCE binary data

```
auto getMyFont() {  
    static const auto result = juce::Typeface::createSystemTypefaceFor(  
        assets::MyFont_ttf, assets::MyFont_ttfSize);  
    return juce::FontOptions{result};  
}  
//...  
label.setFont(getMyFont());
```

Homework

```
git checkout homework/draw-background-noise
```

Thank you!

Where to go from here

- Official JUCE course (free): wolfsoundacademy.com/juce
- DSP course: wolfsoundacademy.com/dsp
 - **40% discount code: CPP026 (valid until May 26)**
- WolfSound blog: thewolfsound.com
- WolfSound YouTube channel: youtube.com/@WolfSoundAudio
- contact@thewolfsound.com